COL 724 Assignment2

Part A

Ankita Singh(2024MCS242891)

Vipul Nayyar(2024CSZ8495)

Server and client implementations have been done in C++ using third-party libraries like RapidJSON[1] for JSON handling, and CLIUtils[2] for command line parsing on server. These two libraries are used in header format, and can be found in the top-level 'include' directory of the source code repository.

The Server creates a socket to listen for new connections. Once it receives a new connection, it reads the packet and passes it to the configured scheduler(FCFS, SJF, RR), to be picked up by a server thread for processing.

Scheduling Policies:

The following scheduling policies have been implemented:

1) **FCFS**(First come First serve): The client request packet is added to the back of a FIFO queue. A mutex lock in C++ is used to synchronize removing a request from the front of the queue. Each available thread waits for a new packet to process using a C++ 'condition variable'. On adding a new packet to the queue, the scheduler notifies one of the available waiting threads. This, request packets are assigned for processing in the order they arrive, as shown in Figure A.

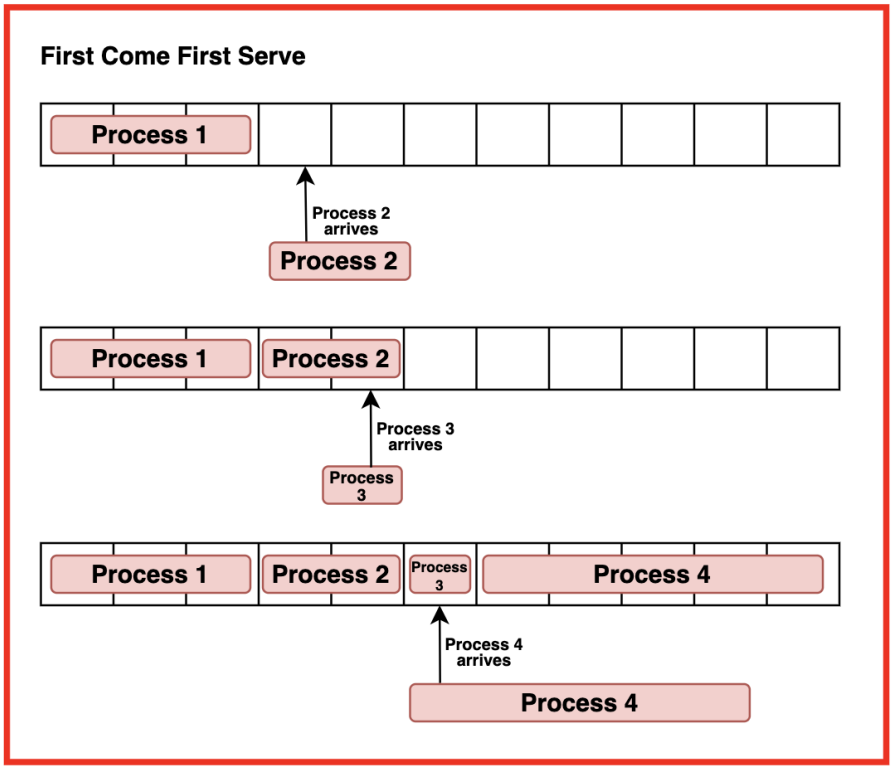


Figure A: First Come First Serve scheduling [3]

2) **SJF**(Shortest Job First – non-preemptive): The insertion order of a request packet in the queue is based on the following two metrics in the following priority order:

   i. **Number of lines to be read/written in a request**: If a request needs to read or write less number of lines, that request is given priority compared to another request that needs to process more number of lines. The assumption taken here is that length of lines to be read/written will always be the same.

   ii. **Arrival Time:** For two packets that will process the same number of lines, the packet that arrived early(like FCFS) will be given priority.

   For every file written, the number of lines present in that file is added to an in-memory cache structure('files_info'), which is queried when a read request is received.

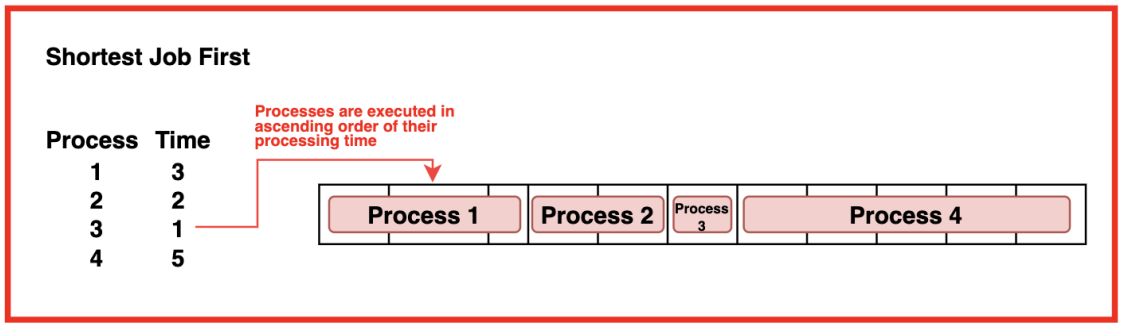


Figure B: Shortest Job First scheduling [3]

3) **RR**(Round-Robin - unweighted): Each request is given a quantum size, which represents that at a time, how much processing time is allocated for that request. Once that quantum expires, the request is inserted in the back of the queue, for another round of scheduling, as shown in Figure C. The incomplete read and write states(file pointers, data read/written) are stored as checkpoints with the request data structure, used to restore the state of the request processing, when it is allowed processing the next time. The quantum size in our experiments is set at 400, that represents 400 ms of execution time.

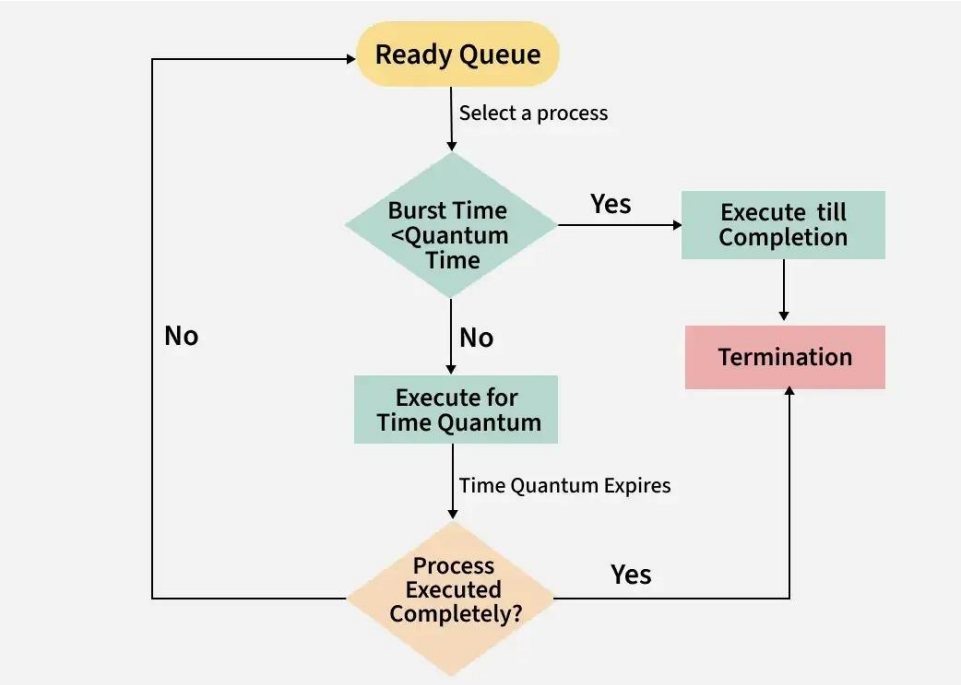


Figure C: Round-robin scheduling [4]

Assumptions:

- JSON format(using RapidJSON) is used to serialize messages between server and client.
- The quantum size for RR is fixed at 400, representing 400 ms of execution slice for each request at a time.
- The processing work for a request is directly proportional to the number of lines that need to be read or written.
- All lines being read or written are assumed to be of the same length.
- The file to be written to the server is sent in one large packet. The write response is a single acknowledgement packet
- Requests and responses use the same 'RequestResponse' data structure in C++ source code.
- The read request for a file is just one packet. But the server responds to the read with multiple packets containing the maximum number of lines allowed by its packetization parameter.

Benchmarking Strategy:

Benchmarking is done using the 'benchmark_script.py' in the '*./bench/*' directory in the source code. For each unique configuration of the system, client workloads are run for 8 seconds concurrently on each client instance. This unique configuration of the whole server-client system is determined by the following variables:

1) **Scheduling policy**: FCFS, SJF, or RR (3 possible values)
2) **Server threads**: 1, 3 or 5 (3 possible values)
3) **Client threads**: 1, 3 or 5 (3 possible values)
4) **Packet Size**: 1, 3 or 5 (3 possible values)

The total possible unique system(server and client) configurations are **81**(3 x 3 x 3 x 3). For each of the above 81 unique configurations, concurrent client workload is sent to server for processing. Each concurrent client runs for 8 seconds and alternatively reads and writes the same sample file to the server.

The number of operations compeleted by each client during this period is logged to a log file with the format
“{sched_policy}_clients{client_threads}_servers{server_threads}_p{packet_size}.csv”

For each of the above system configuration, the following 4 values are computed from the logged operations:

1) Mean response time
2) Median response time
3) Mean waiting time
4) Throughput(ops/s)

When the benchmark is completed, all the individual csv files are combined to generate graphs. The above computed values act as Y-axis, and a pair of variables described above are used for X-axis and different lines in the same graph. A total of **48**(4 x 4 x 3) graphs are generated.

Data Processing and Analysis:

To automate the analysis, we developed two Python scripts

1. **completedata_gen.py** — prepares and augments raw client logs
2. **plot_experiments.py** — aggregates and visualizes experimental results

These scripts form a pipeline to transform raw experimental logs to meaningful performance graphs which we used for performance analysis.

1. Data Preparation Script (**completedata_gen.py)**
   The client records per-request timestamps and configuration metadata in a csv file. The script computes Response Time and Waiting Time and append these two new columns to the dataset and saves as complete_data.csv
2. Plot Generation Script (**plot_experiments.py**)
   This script performs aggregation, metric computation and visualization. It produces plots that show how different systems parameters affect key performance metrics.

   Data Aggregation:

   Groups data by (sched_policy, server_threads, client_threads, packet_size) and computes:
   Mean and median response times
   Mean waiting time
   Throughput (requests/sec)

   Plot Generation:

   Creates plots for each metric — Mean Response Time, Median Response Time, Mean Waiting Time, and Throughput.
   In each plot:
   One variable is on the X-axis
   One varies across lines (legend)
   The remaining two are automatically fixed and shown in captions and filenames

Output:

All figures are saved in the plots/ directory with descriptive names, e.g. server_threads_vs_mean_response_time_diff_sched_policy_fixed_client_threads2_packet_size4.png

Experimental Setup

In this experiment, we evaluate the performance of the client–server system by varying four key configuration parameters: **client thread count**, **server thread count**, **packet size**, and **scheduling policy**

For each configuration, performance metrics are collected and plotted on the **Y-axis**, including:

- **Mean and Median Response Time (ms)** — total time from request dispatch to completion.
- **Mean Waiting Time (ms)** — time a request waits in the queue before being served.
- **Throughput (requests/sec)** — number of requests completed per second, indicating overall system efficiency.

Results:

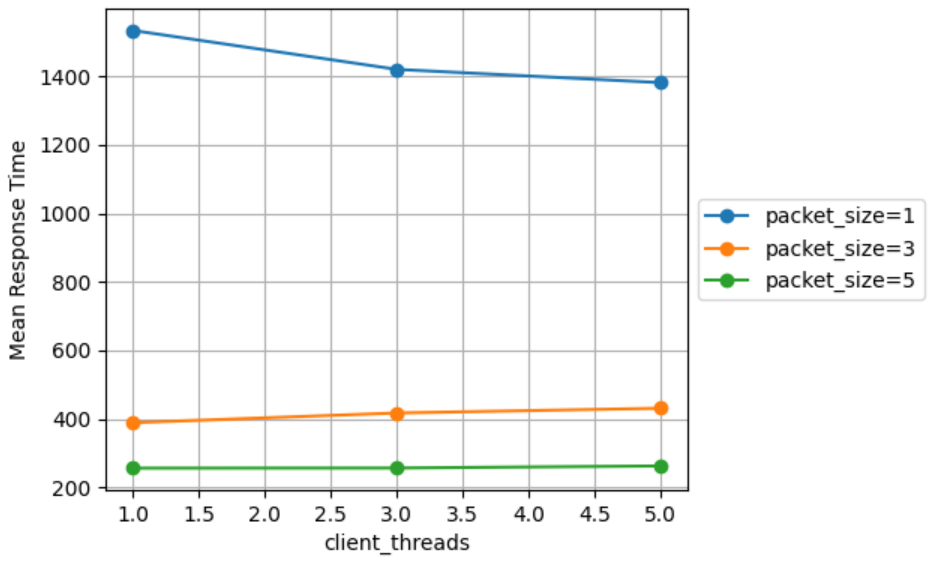


Figure 9: client_threads vs mean_response_time for different packet_size (fixed sched_policy=fcfs, server_threads=1)

The above graph plots **client_threads vs. mean_response_time**, with **packet_size** varied to observe its impact. The **scheduling policy** is fixed to **FCFS**, and the **server_thread count** is set to **1**. The plot indicates that as **packet size increases**, the **mean response time decreases**, suggesting that larger packet sizes lead to more efficient transmission and reduced overhead under this configuration.

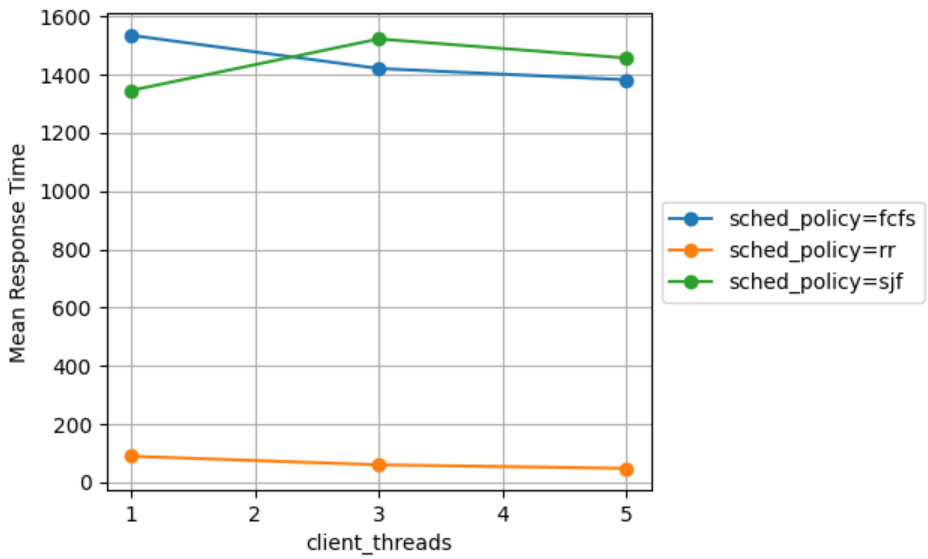


Figure 7: client_threads vs mean_response_time for different sched_policy (fixed server_threads=1, packet_size=1)

In the same way the above plot shows that round robin policy gives better response time as response time is least for RR among three sched policy. This indicates that RR provides better load distribution and leading to improved responsiveness compared to **FCFS** and **SJF**.

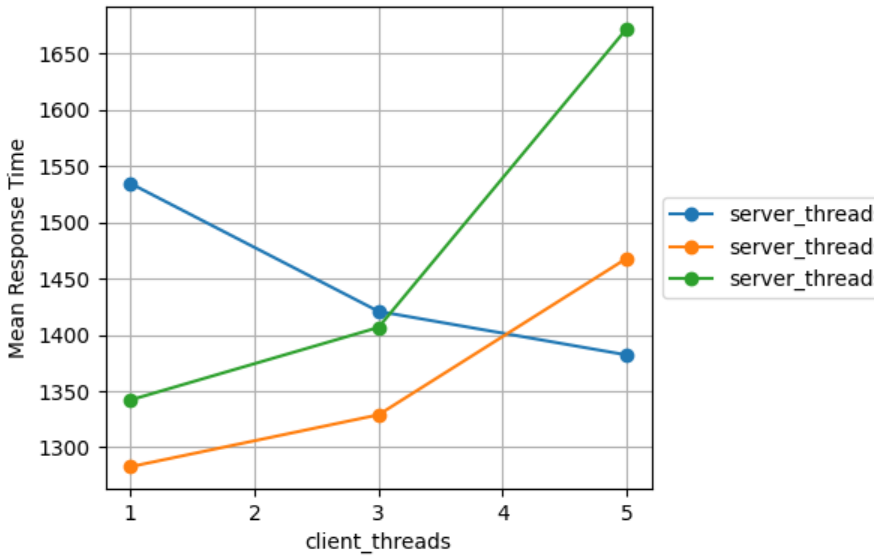


Figure 8: client_threads vs mean_response_time for different server_threads (fixed sched_policy=fcfs, packet_size=1)

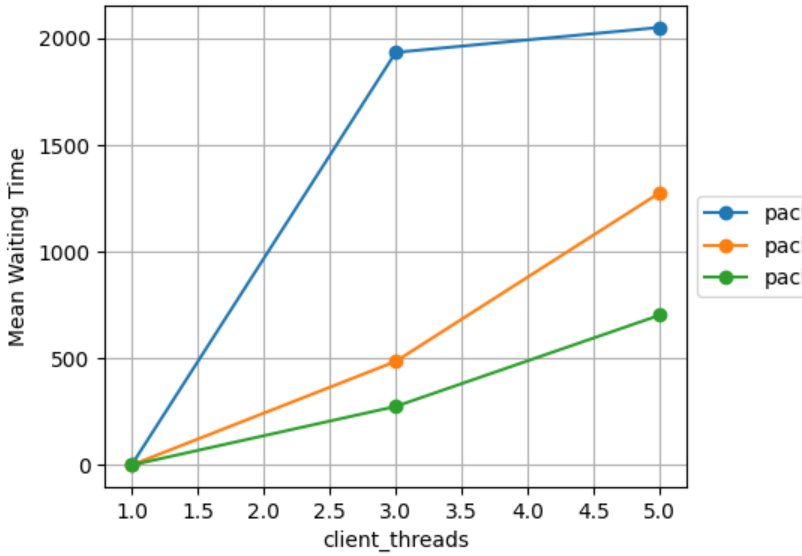


Figure 33: client_threads vs mean_waiting_time for different packet_size (fix
sched_policy=fcfs, server_threads=1)

From above plot, It can be observed that for **small packet sizes** and **high client thread counts**, the **waiting time is highest**, indicating that FCFS suffers from increased queuing delays under heavy client concurrency and smaller request sizes.

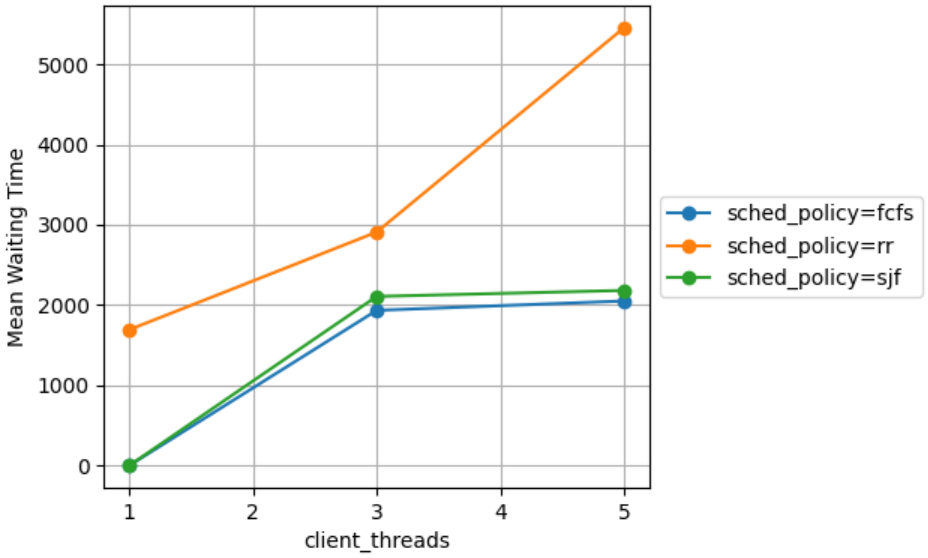


Figure 31: client_threads vs mean_waiting_time for different sched_policy (fixed
server_threads=1, packet_size=1)

It shows that at **high client thread counts**, the **Round Robin (RR)** policy exhibits the **highest waiting time** among the three policies, indicating reduced performance under heavy concurrency compared to **FCFS** and **SJF**.

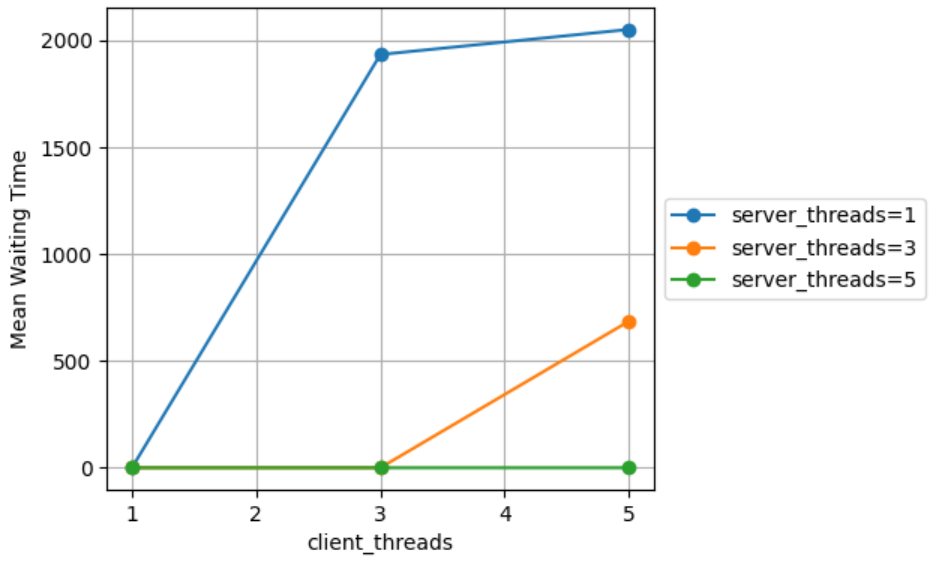


Figure 32: client_threads vs mean_waiting_time for different server_threads (fixed sched_policy=fcfs, packet_size=1)

It shows that increasing the **number of server threads** reduces the **mean waiting time**, while configurations with fewer server threads result in the **highest waiting times**.

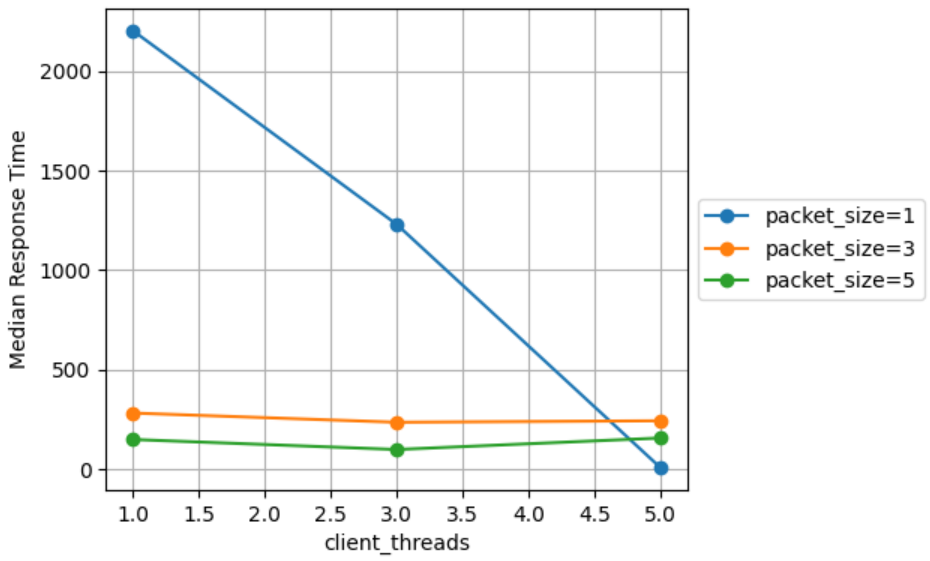


Figure 21: client_threads vs median_response_time for different packet_size (fixed sched_policy=fcfs, server_threads=1)

As the number of client threads increases, the **response time decreases rapidly**, approaching zero, indicating that higher client concurrency improves request processing efficiency under this configuration.

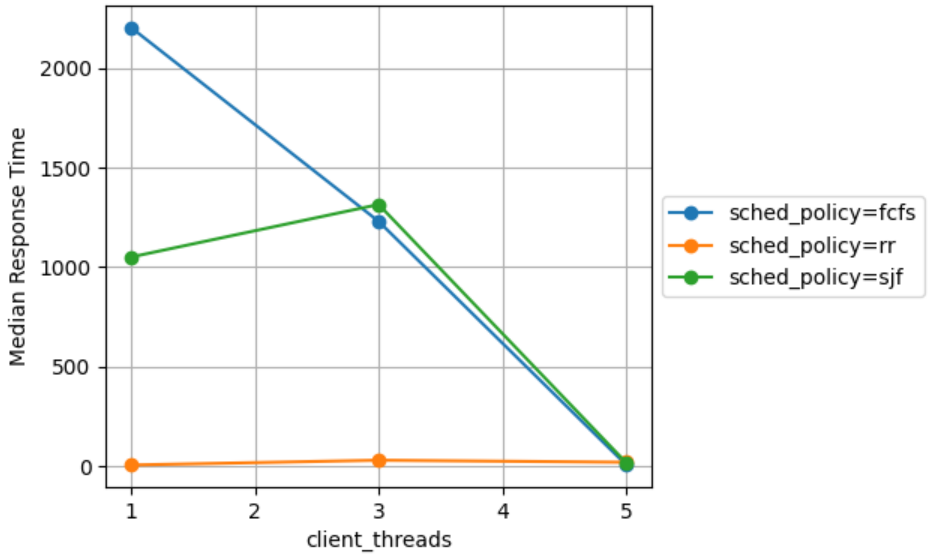


Figure 19: client_threads vs median_response_time for different sched_policy (fixed server_threads=1, packet_size=1)

It can be observed that **Round Robin (RR)** consistently achieves the **lowest response time** regardless of client thread count, outperforming **FCFS** and **SJF**

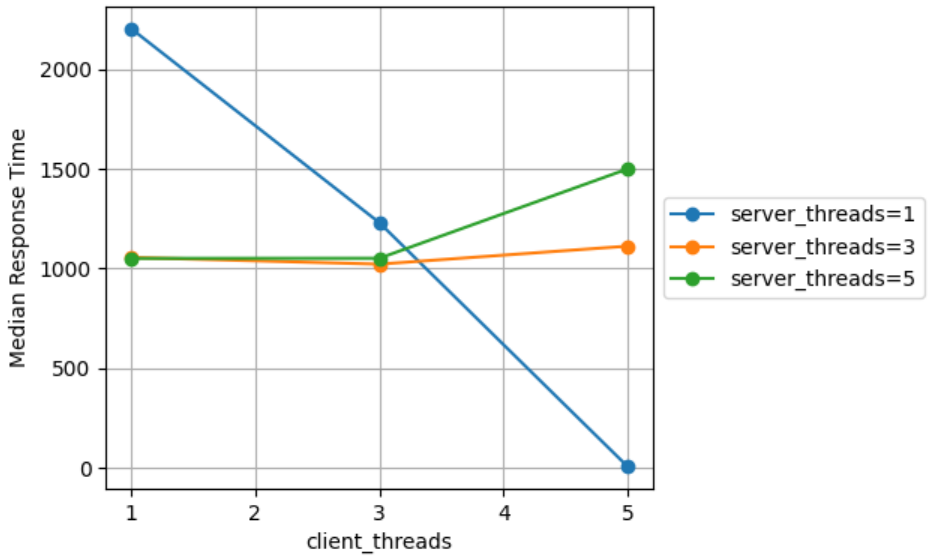


Figure 20: client_threads vs median_response_time for different server_threads (fixed sched_policy=fcfs, packet_size=1)

It shows that when the **server thread count is low**, the **response time initially starts high** but **decreases** as the number of client threads increases.

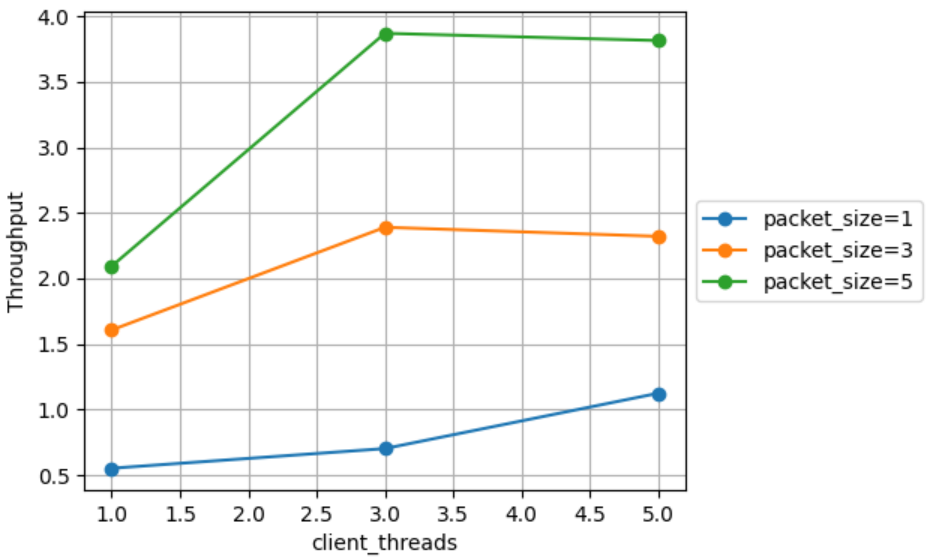


Figure 45: client_threads vs throughput for different packet_size (fixed sched_policy=fcfs, server_threads=1)

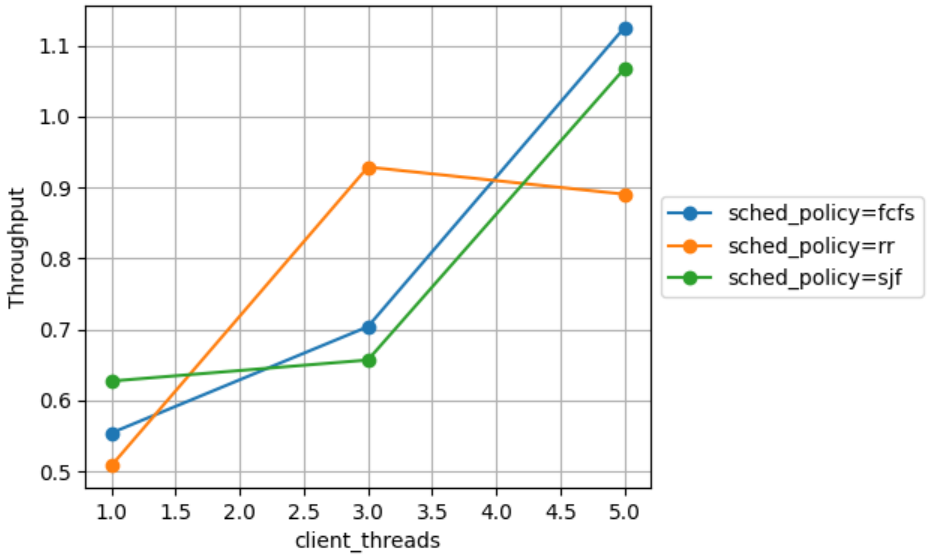


Figure 43: client_threads vs throughput for different sched_policy (fixed server_threads=1, packet_size=1)

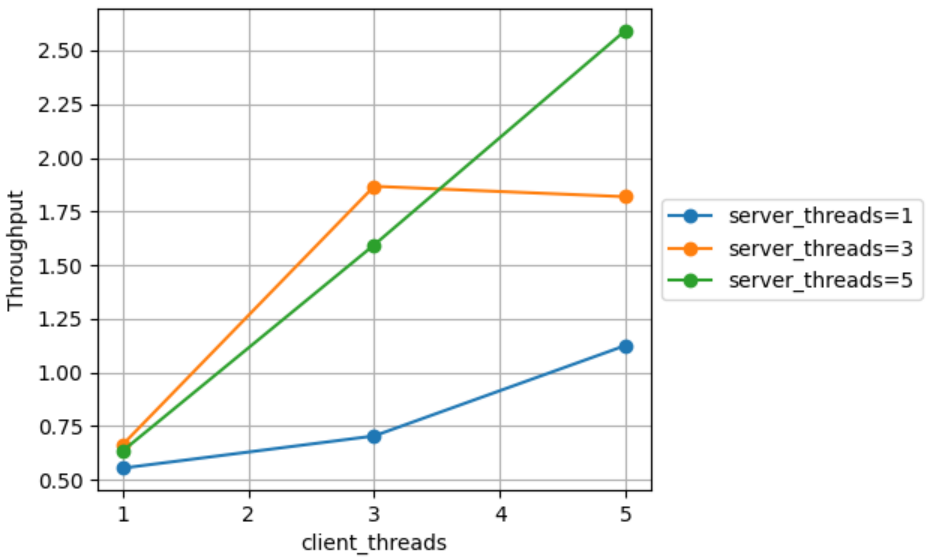


Figure 44: client_threads vs throughput for different server_threads (fixed sched_policy=fcfs, packet_size=1)

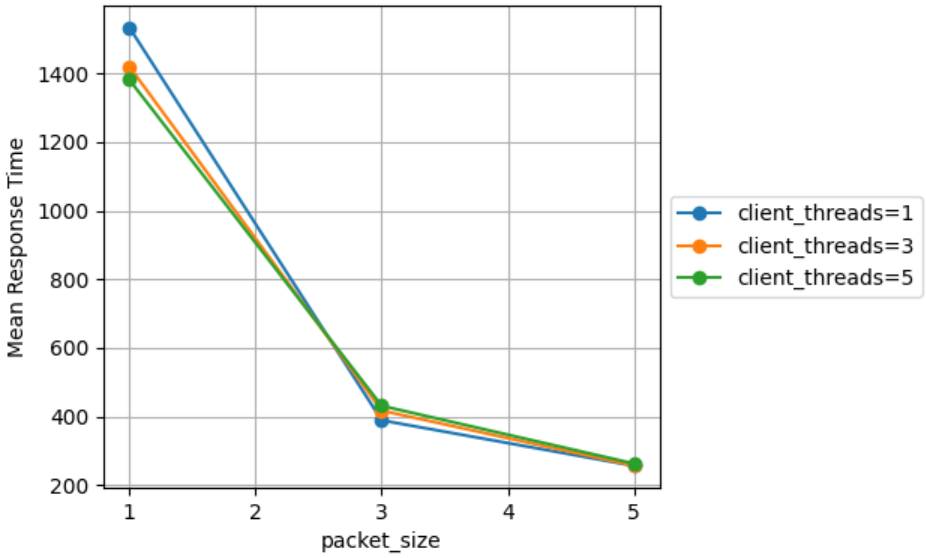


Figure 12: packet_size vs mean_response_time for different client_threads (fixed sched_policy=fcfs, server_threads=1)

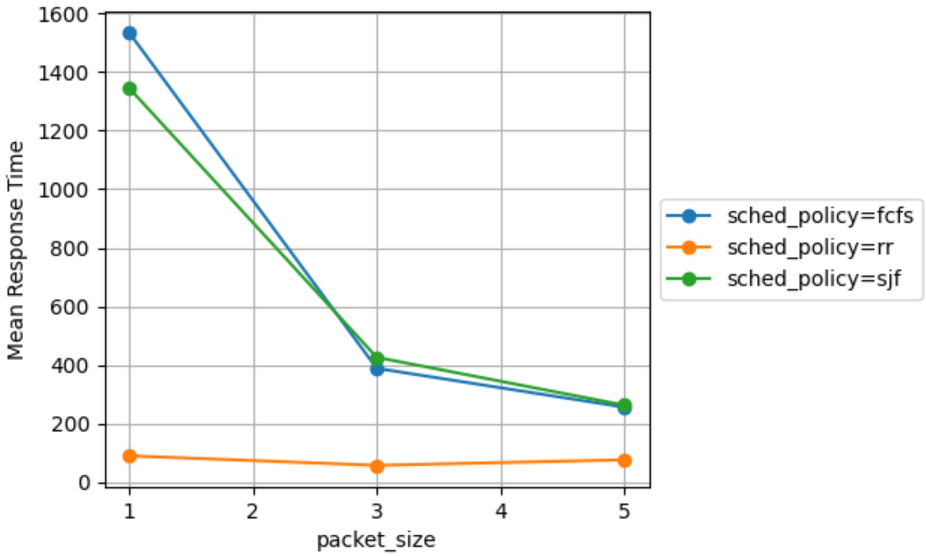


Figure 10: packet_size vs mean_response_time for different sched_policy (fixed server_threads=1, client_threads=1)

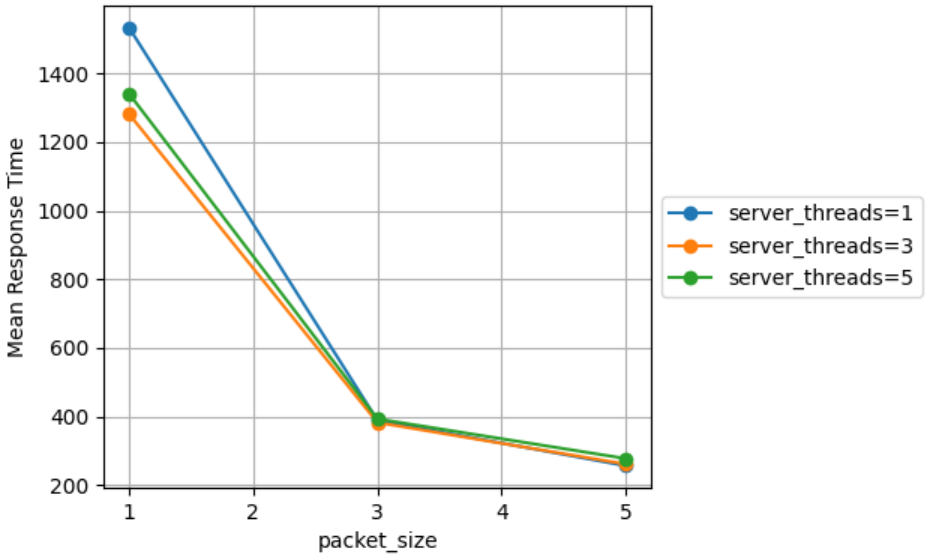


Figure 11: packet_size vs mean_response_time for different server_threads (fixed sched_policy=fcfs, client_threads=1)

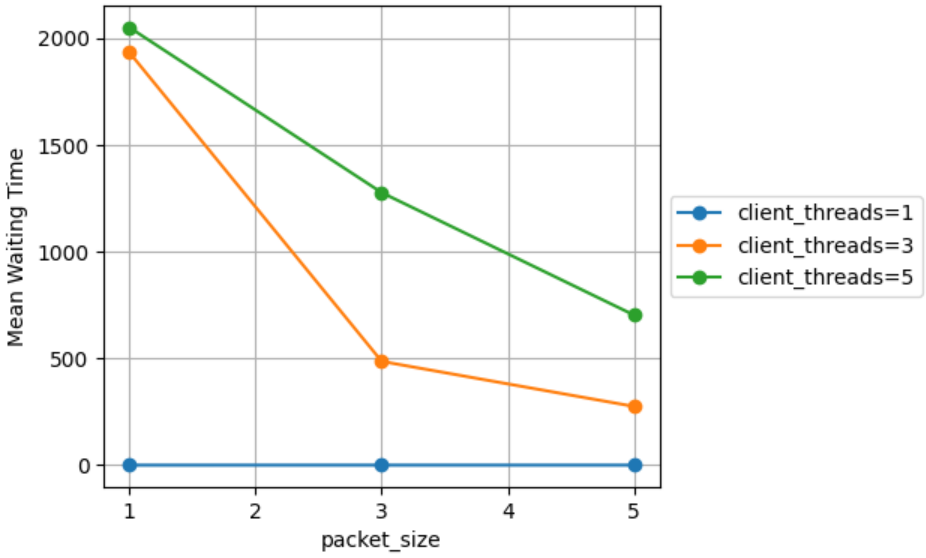


Figure 36: packet_size vs mean_waiting_time for different client_threads (fixed sched_policy=fcfs, server_threads=1)

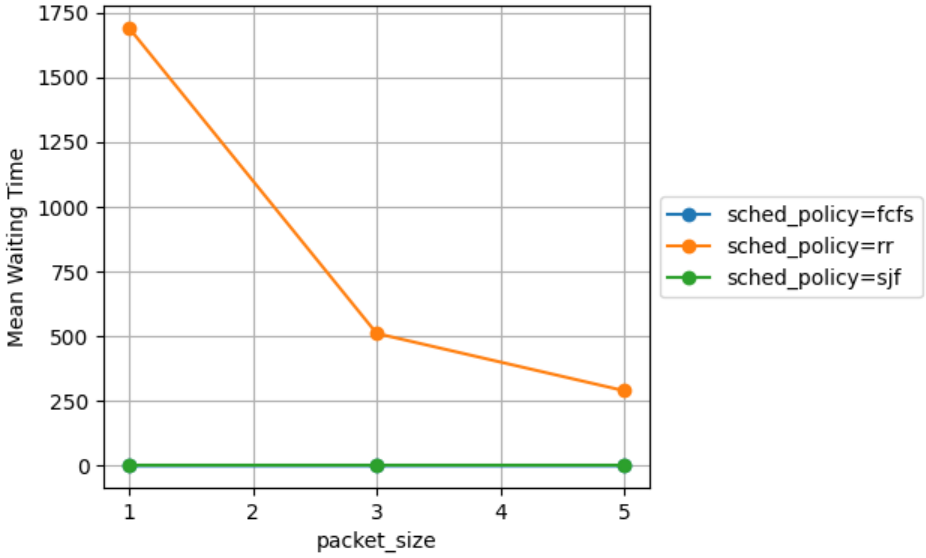


Figure 34: packet_size vs mean_waiting_time for different sched_policy (fixed server_threads=1, client_threads=1)

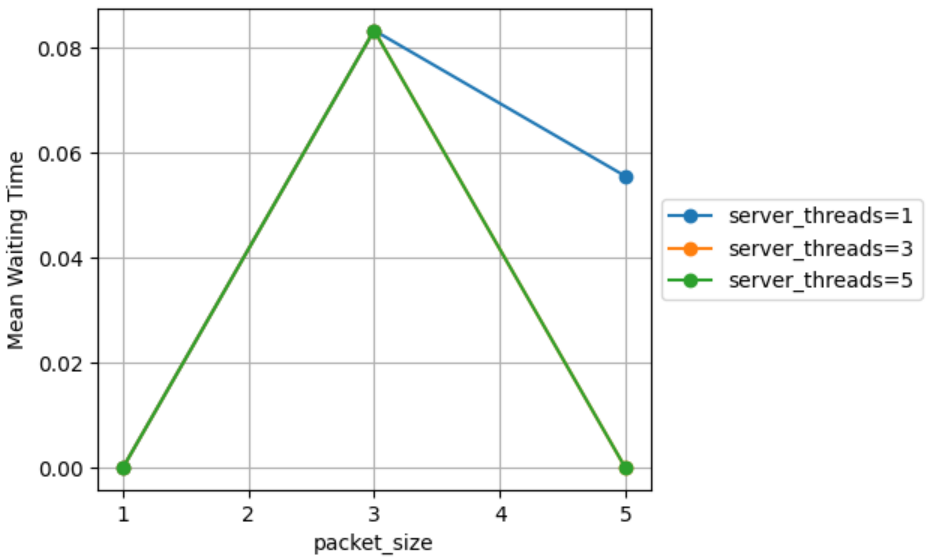


Figure 35: packet_size vs mean_waiting_time for different server_threads (fixed sched_policy=fcfs, client_threads=1)

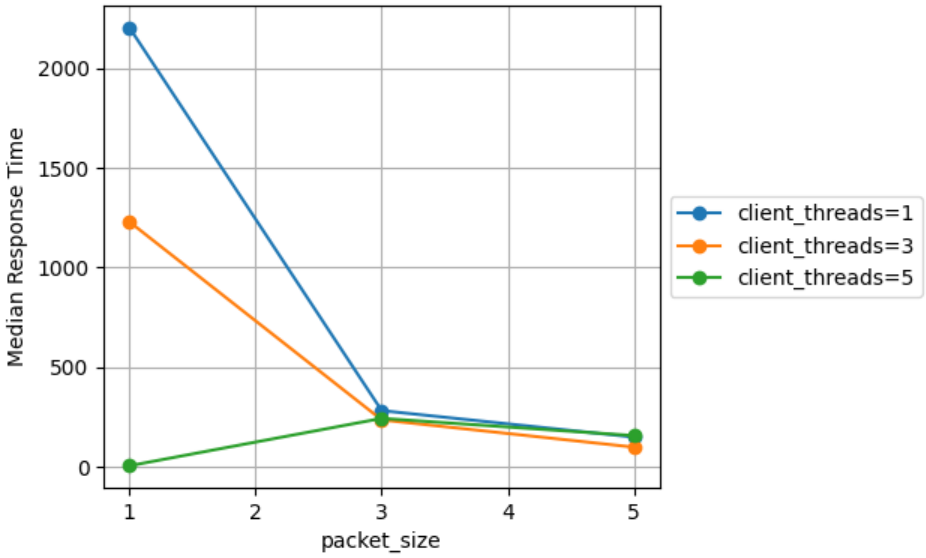


Figure 24: packet_size vs median_response_time for different client_threads (fixed sched_policy=fcfs, server_threads=1)

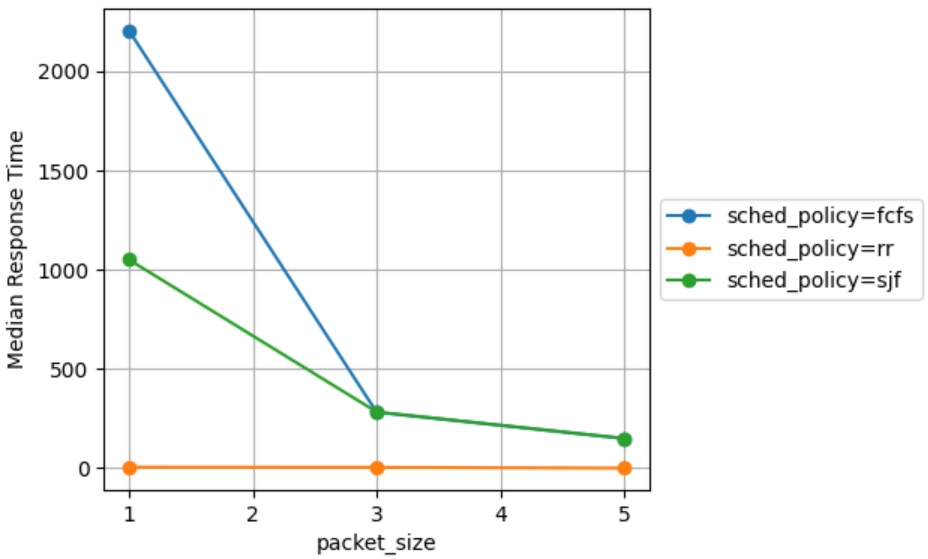


Figure 22: packet_size vs median_response_time for different sched_policy (fixed server_threads=1, client_threads=1)

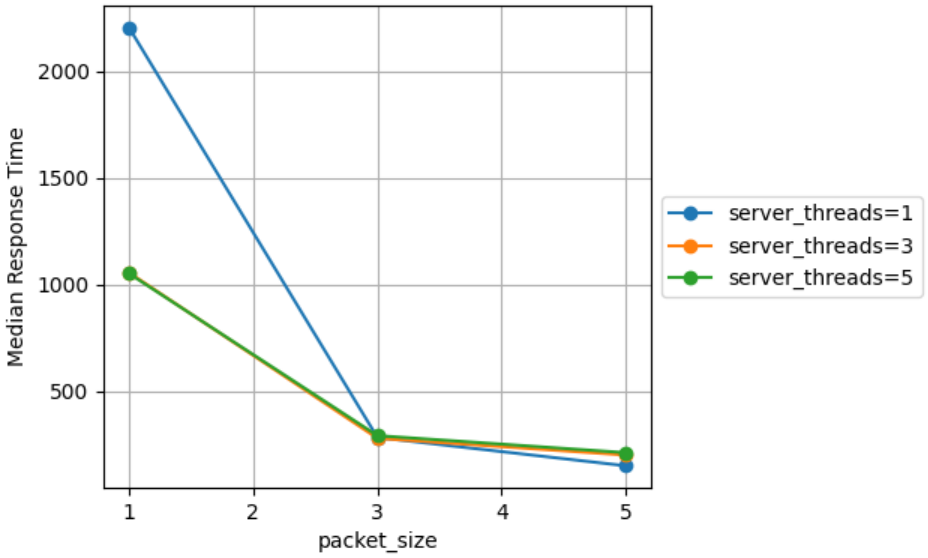


Figure 23: packet_size vs median_response_time for different server_threads (fixed sched_policy=fcfs, client_threads=1)

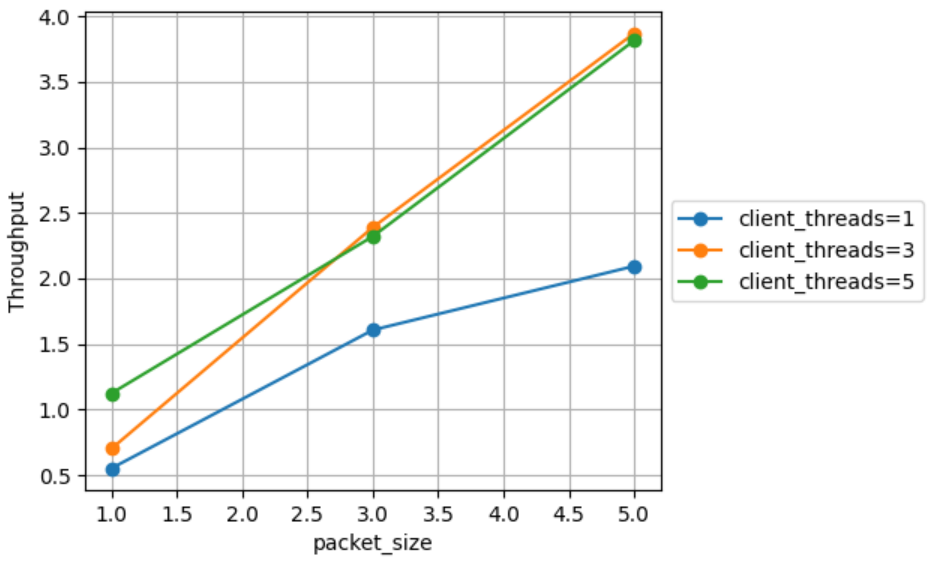


Figure 48: packet_size vs throughput for different client_threads (fixed sched_policy=fcfs, server_threads=1)

It indicates that configurations with **more client threads** and **larger packet sizes** achieve **higher throughput**, suggesting that higher concurrency combined with larger requests improves overall system efficiency under FCFS scheduling.

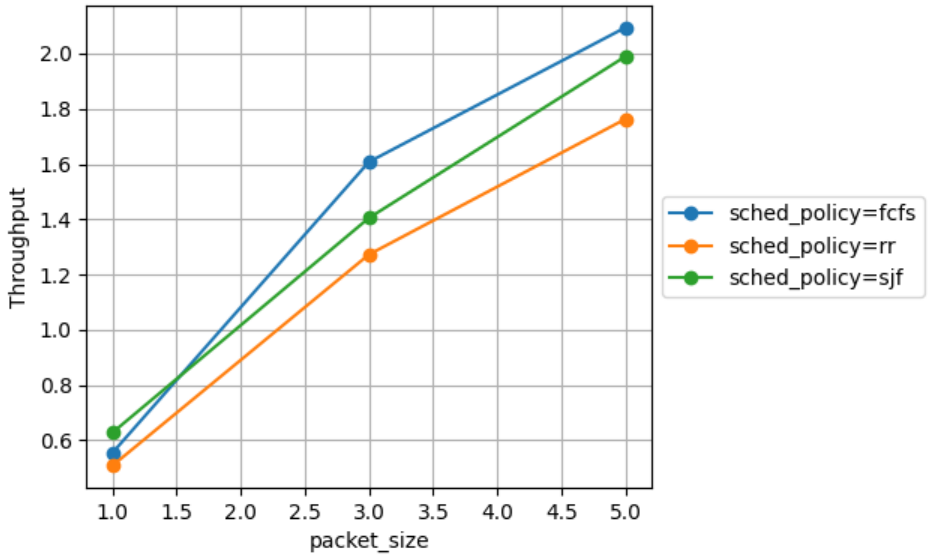


Figure 46: packet_size vs throughput for different sched_policy (fixed server_threads=1, client_threads=1)

It shows that **FCFS** achieves **better throughput** than the other policies, and that **throughput increases** as the **packet size increases**

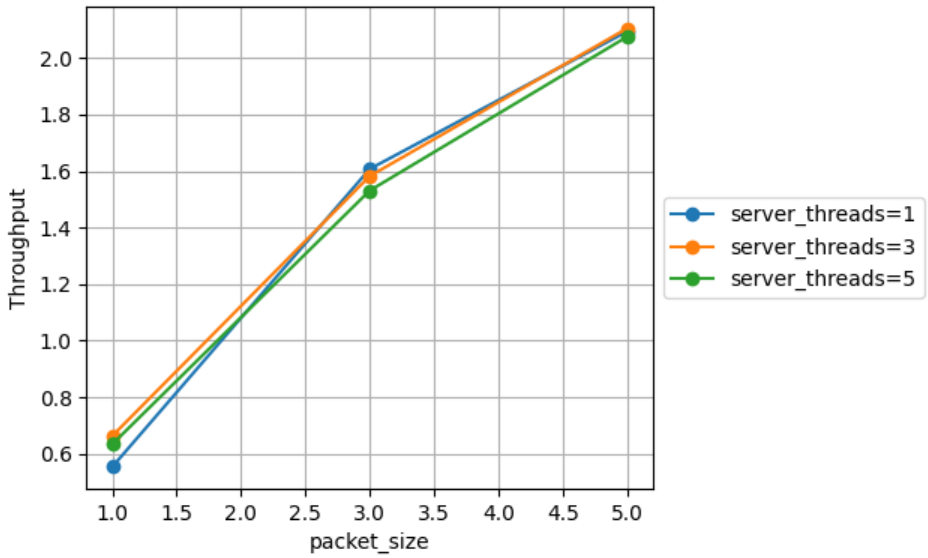


Figure 47: packet_size vs throughput for different server_threads (fixed sched_policy=fcfs, client_threads=1)

It can be observed that as the **packet size increases**, the **throughput steadily increases** and produces **similar results regardless of the server thread count**.

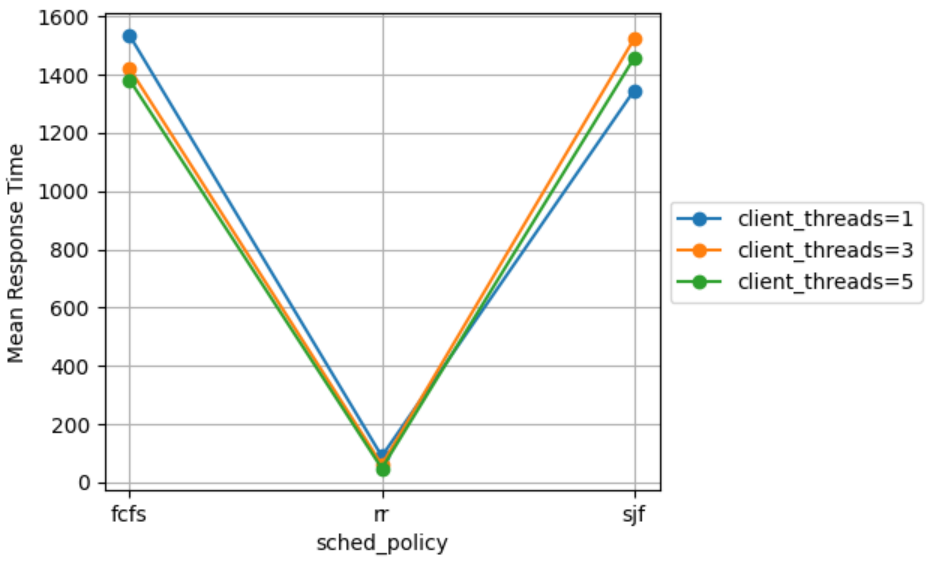


Figure 2: sched_policy vs mean_response_time for different client_threads (fixed server_threads=1, packet_size=1)

It shows that **Round Robin (RR)** consistently provides the **lowest response time**, while **FCFS** and **SJF** exhibit significantly **higher response times**.

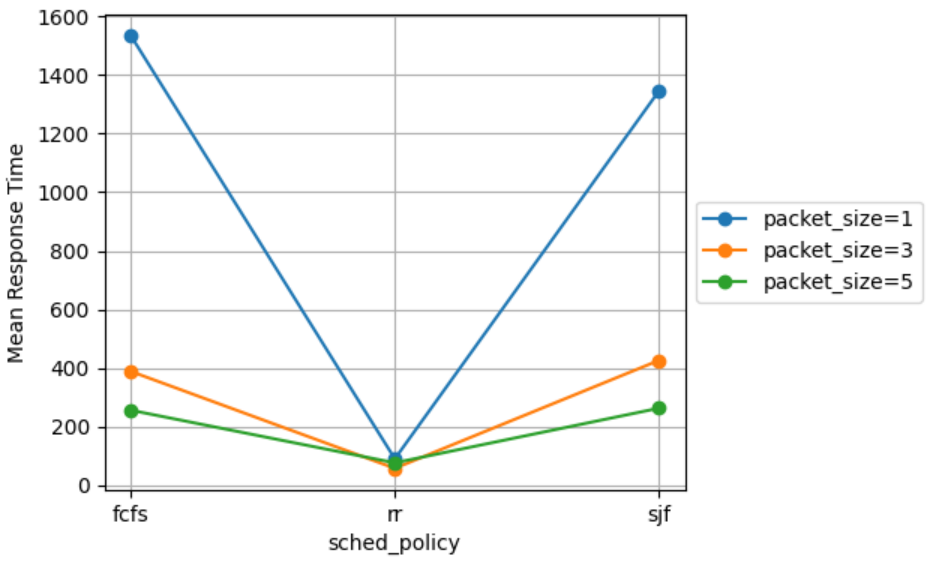


Figure 3: sched_policy vs mean_response_time for different packet_size (fixed server_threads=1, client_threads=1)

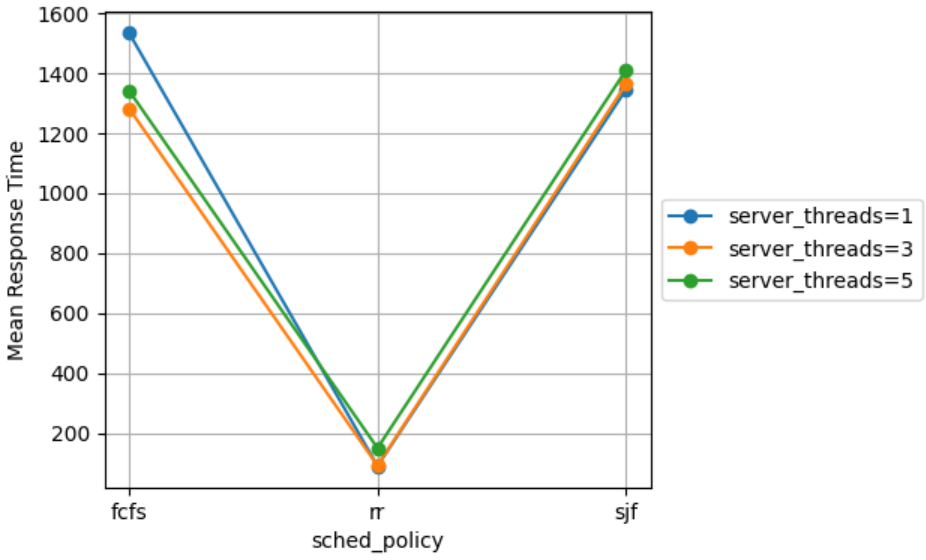


Figure 1: sched_policy vs mean_response_time for different server_threads (fixed client_threads=1, packet_size=1)

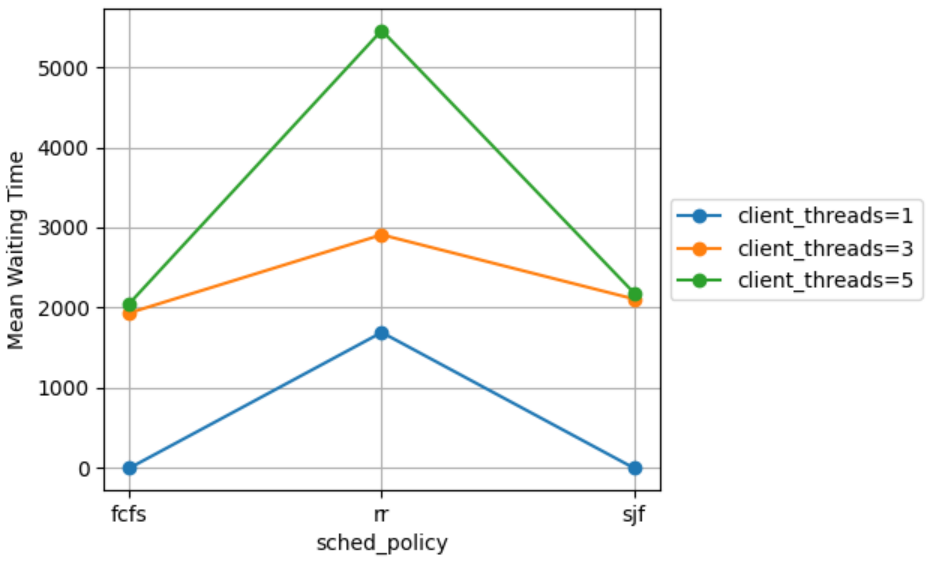


Figure 26: sched_policy vs mean_waiting_time for different client_threads (fixed server_threads=1, packet_size=1)

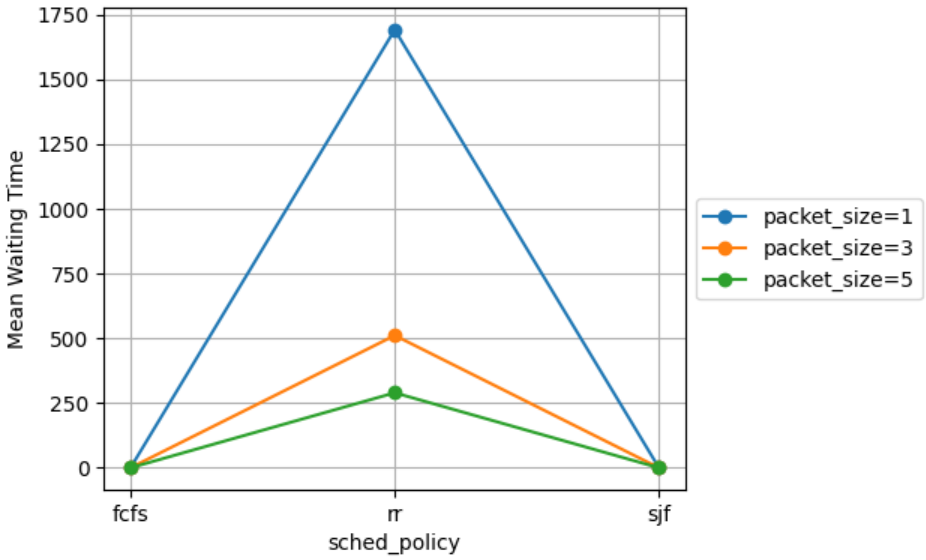


Figure 27: sched_policy vs mean_waiting_time for different packet_size (fixed server_threads=1, client_threads=1)

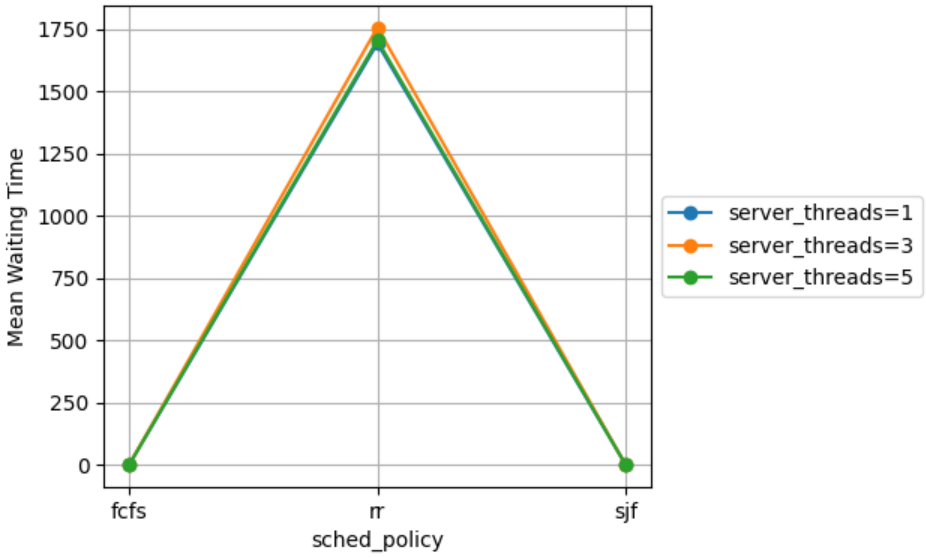


Figure 25: sched_policy vs mean_waiting_time for different server_threads (fixed client_threads=1, packet_size=1)

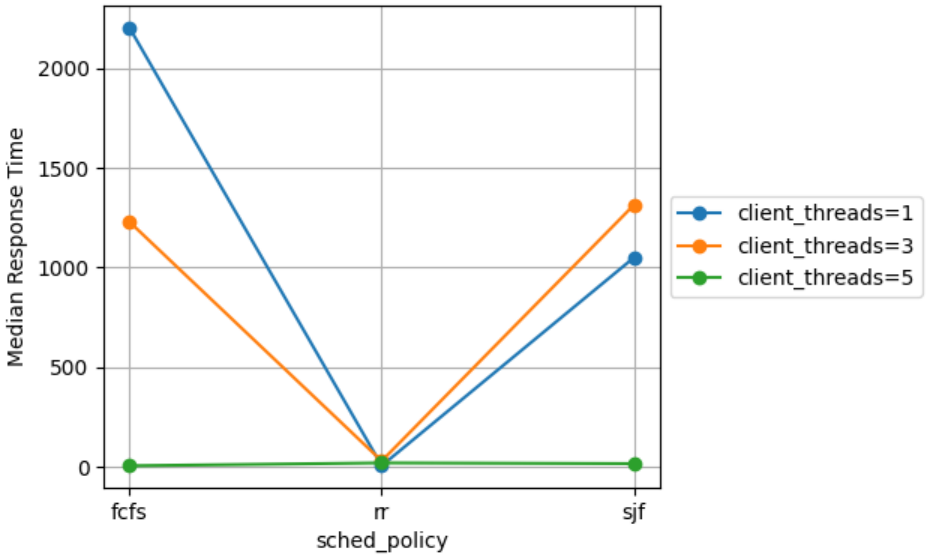


Figure 14: sched_policy vs median_response_time for different client_threads (fixed server_threads=1, packet_size=1)

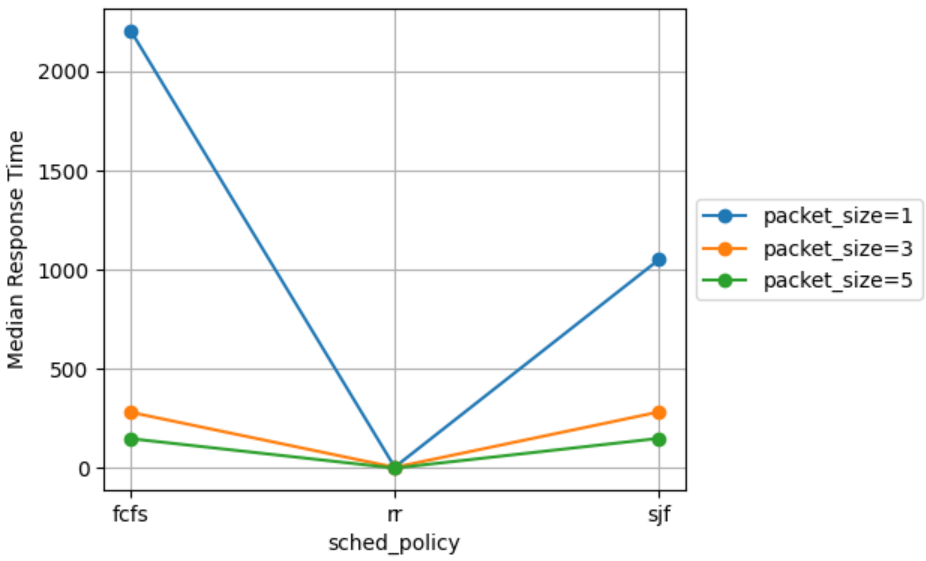


Figure 15: sched_policy vs median_response_time for different packet_size (fixed server_threads=1, client_threads=1)

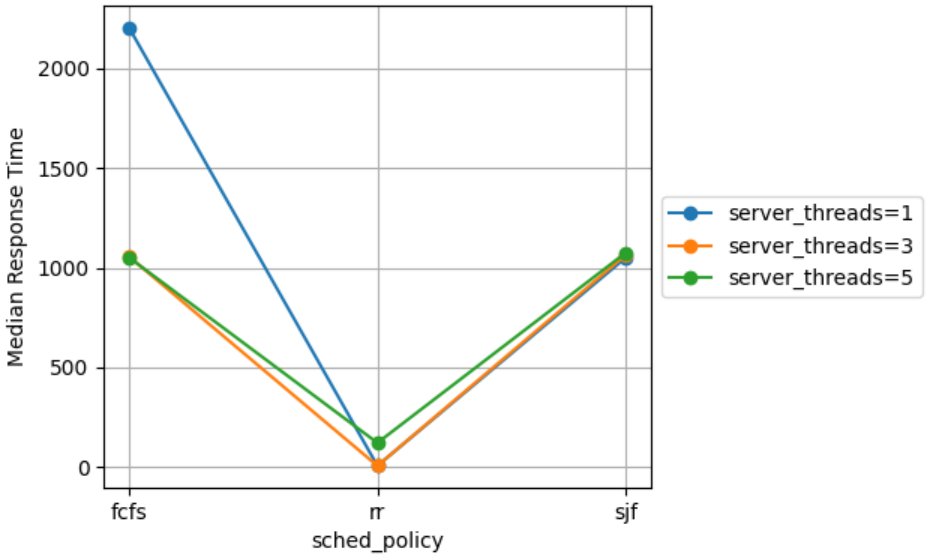


Figure 13: sched_policy vs median_response_time for different server_threads (fixed client_threads=1, packet_size=1)

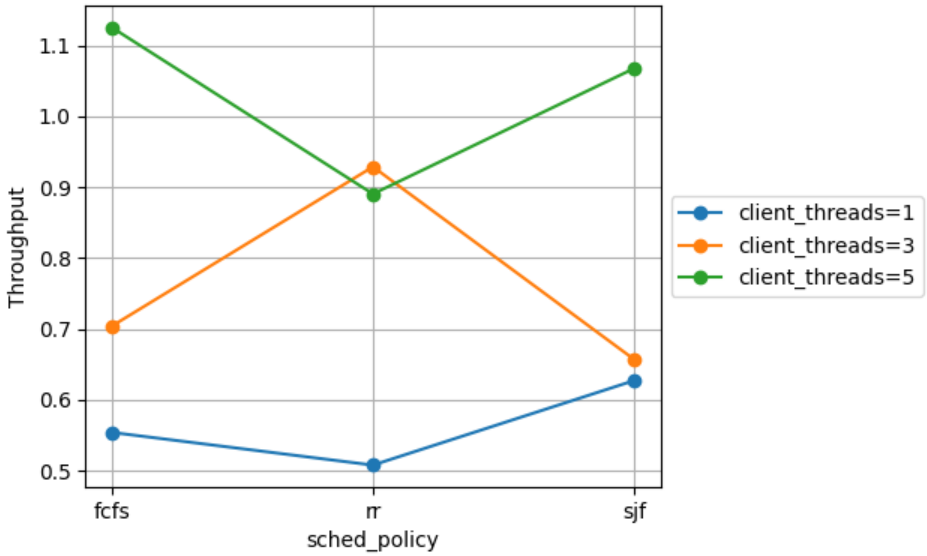


Figure 38: sched_policy vs throughput for different client_threads (fixed server_threads=1, packet_size=1)

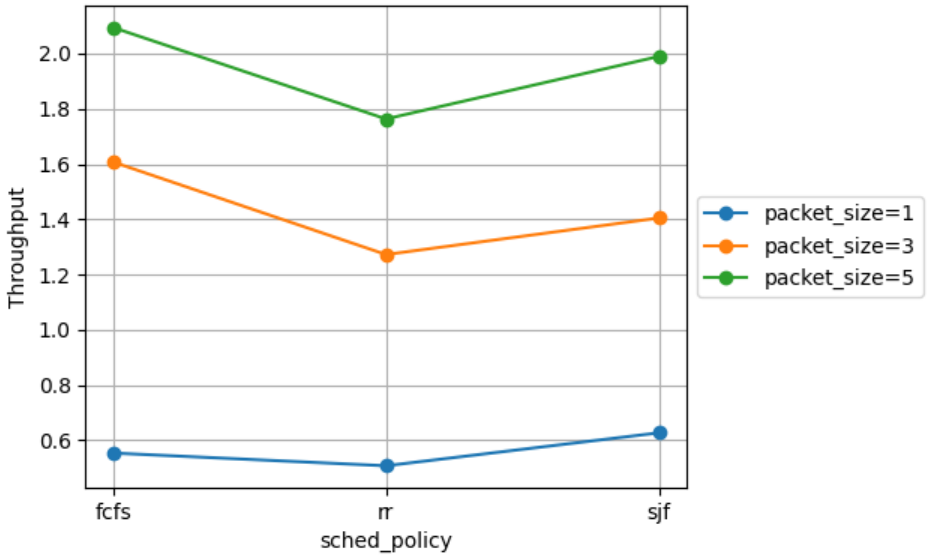


Figure 39: sched_policy vs throughput for different packet_size (fixed server_threads=1, client_threads=1)

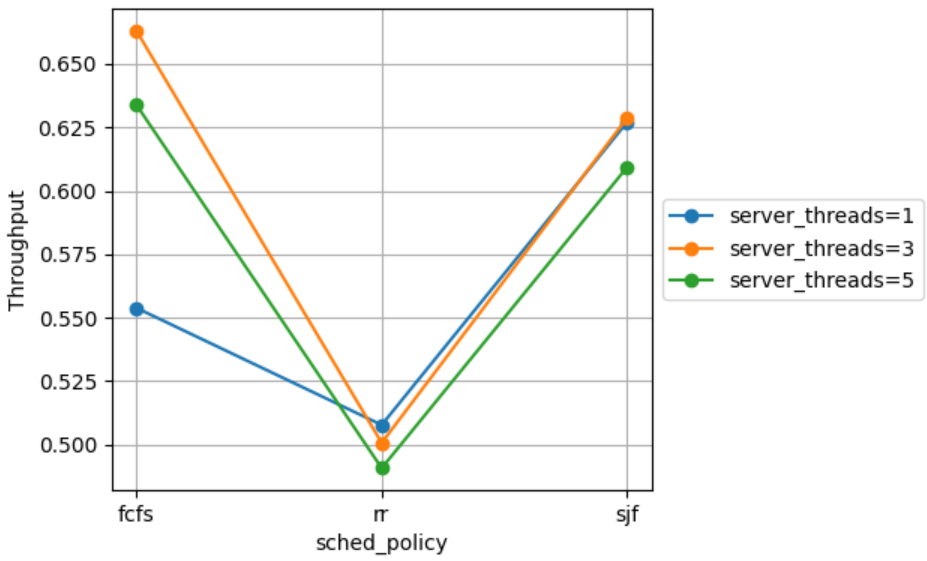


Figure 37: sched_policy vs throughput for different server_threads (fixed client_threads=1, packet_size=1)

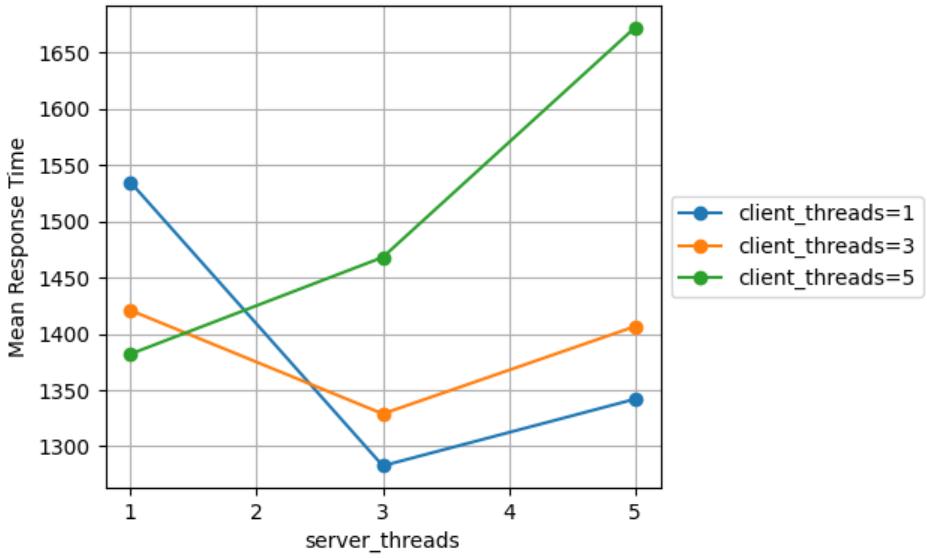


Figure 5: server_threads vs mean_response_time for different client_threads (fixed sched_policy=fcfs, packet_size=1)

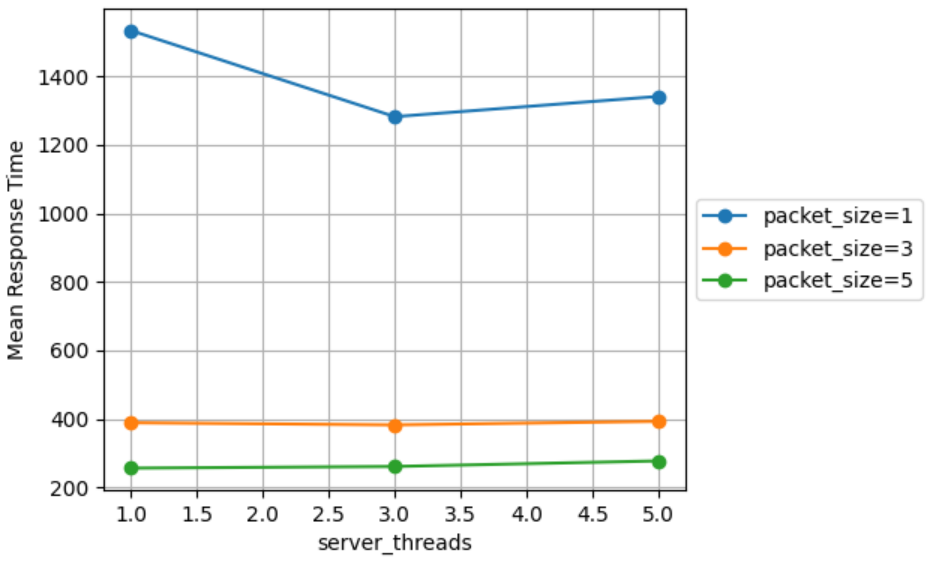


Figure 6: server_threads vs mean_response_time for different packet_size (fixed sched_policy=fcfs, client_threads=1)

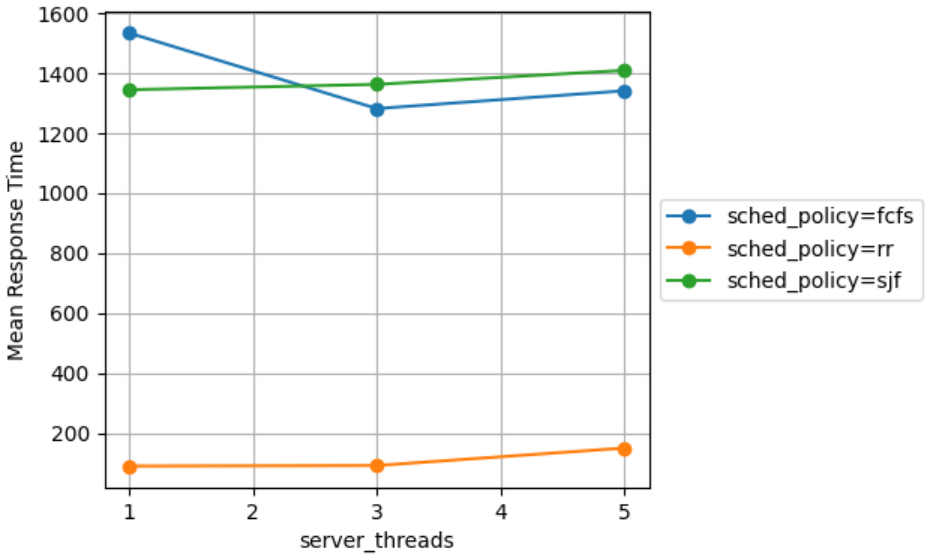


Figure 4: server_threads vs mean_response_time for different sched_policy (fixed client_threads=1, packet_size=1)

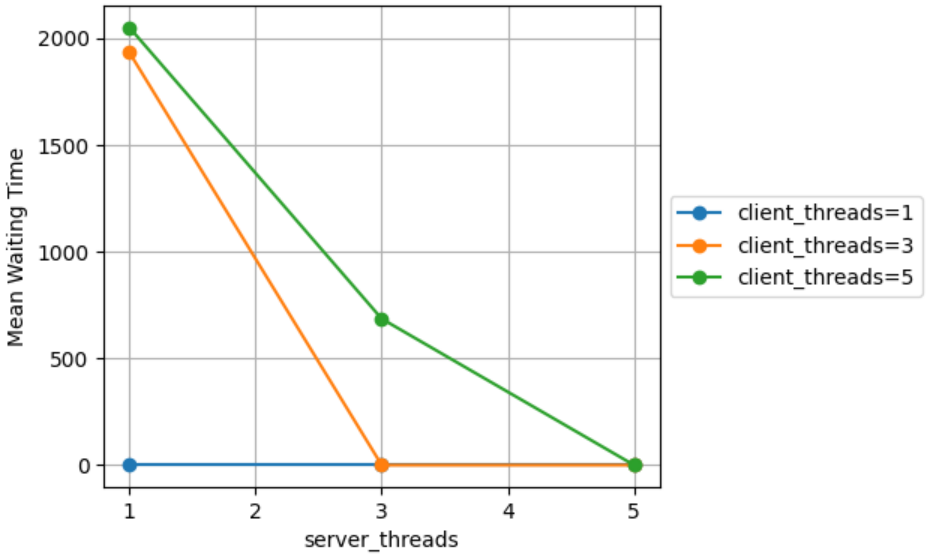


Figure 29: server_threads vs mean_waiting_time for different client_threads (fixed sched_policy=fcfs, packet_size=1)

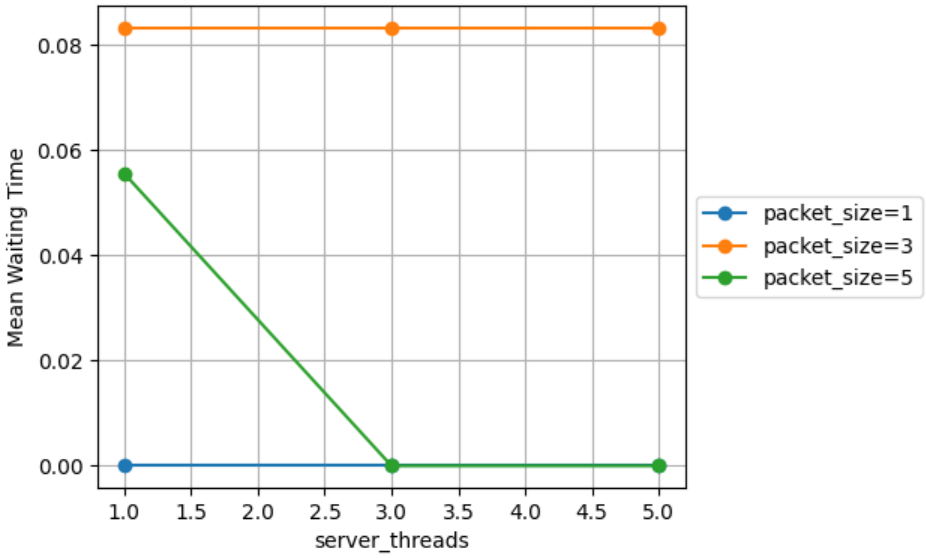


Figure 30: server_threads vs mean_waiting_time for different packet_size (fixed sched_policy=fcfs, client_threads=1)

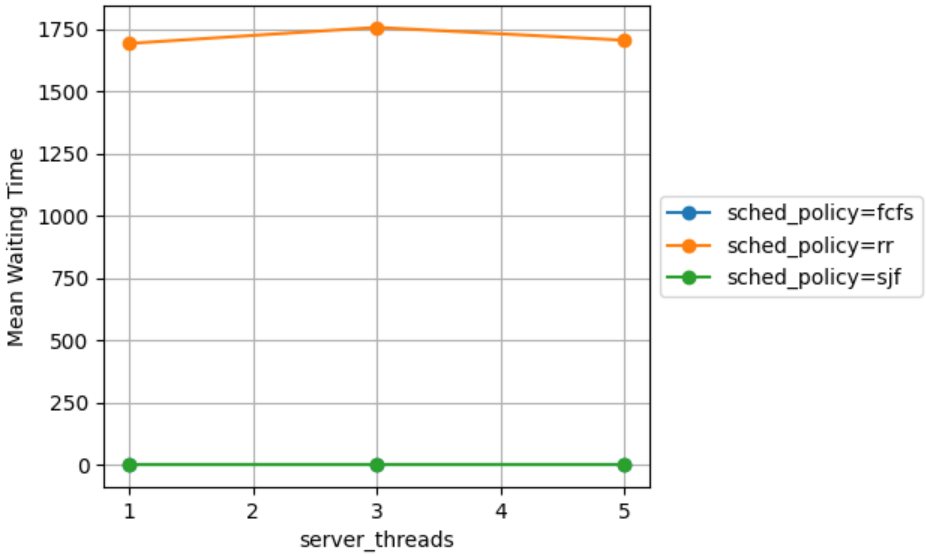


Figure 28: server_threads vs mean_waiting_time for different sched_policy (fixed client_threads=1, packet_size=1)

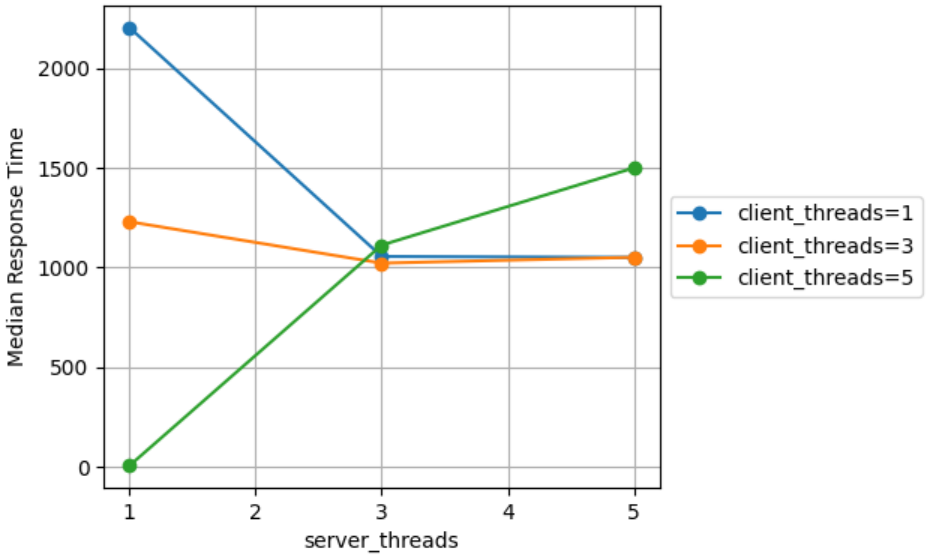


Figure 17: server_threads vs median_response_time for different client_threads (fixed sched_policy=fcfs, packet_size=1)

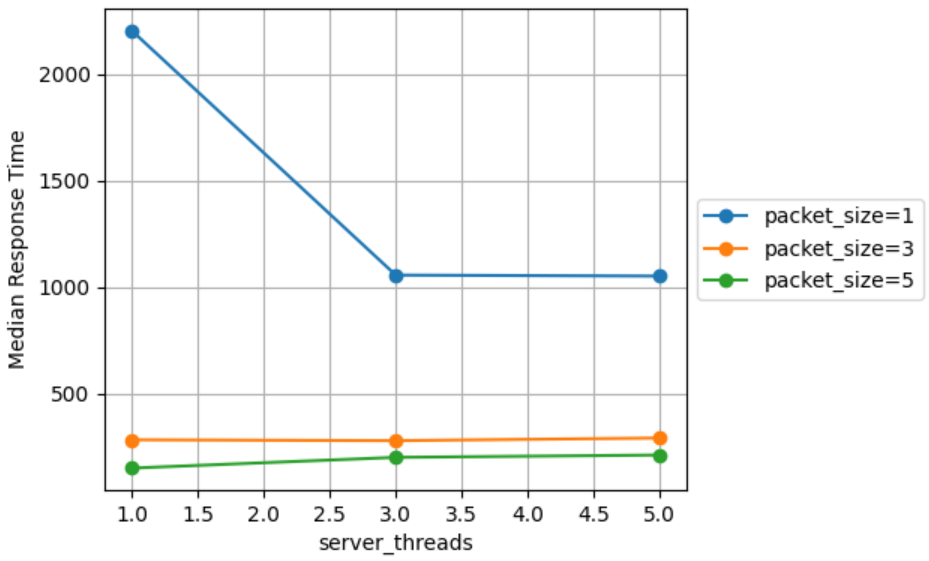


Figure 18: server_threads vs median_response_time for different packet_size (fixed sched_policy=fcfs, client_threads=1)

It can be observed that **increasing the number of server threads** reduces the **response time**, and the effect is more visible for **large packet sizes** compared to small packet sizes.

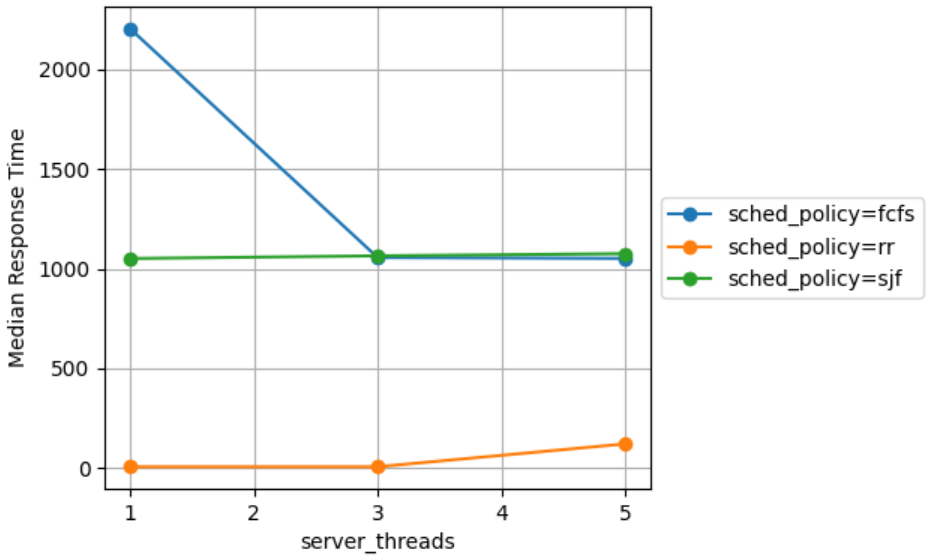


Figure 16: server_threads vs median_response_time for different sched_policy (fixed client_threads=1, packet_size=1)

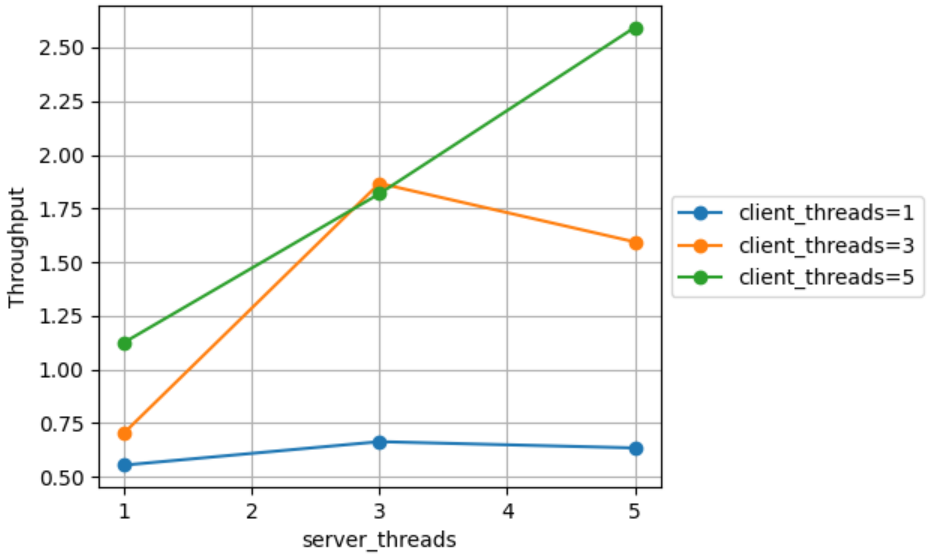


Figure 41: server_threads vs throughput for different client_threads (fixed sched_policy=fcfs, packet_size=1)

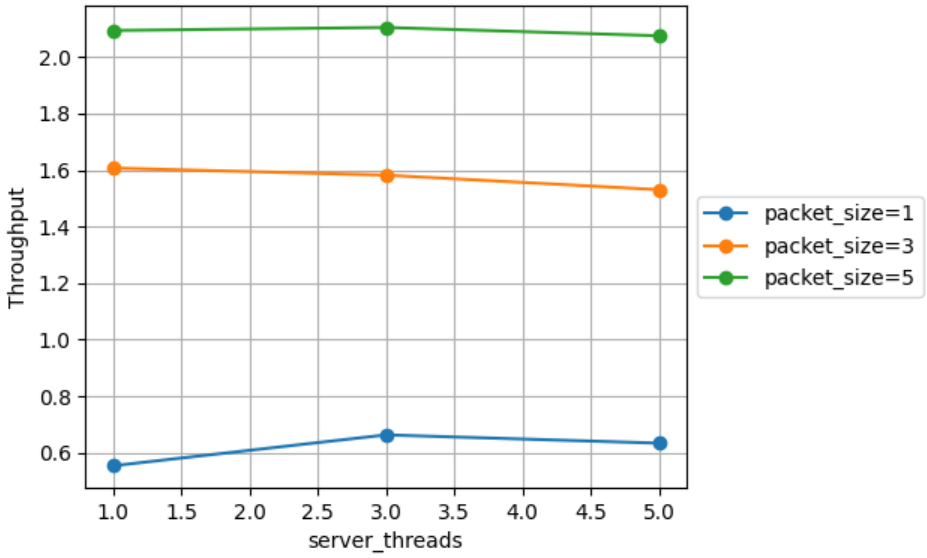


Figure 42: server_threads vs throughput for different packet_size (fixed sched_policy=fcfs, client_threads=1)

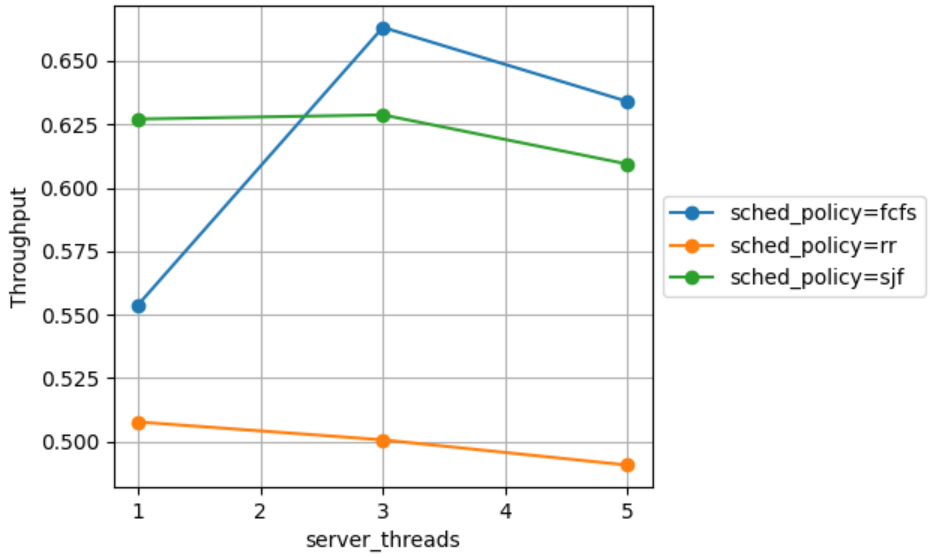


Figure 40: server_threads vs throughput for different sched_policy (fixed client_threads=1, packet_size=1)

Result Analysis:

The experiments demonstrate how client and server configurations, packet size, and scheduling policy impact system performance.

Round Robin (RR) consistently provides the lowest **response time**, especially for small packet sizes and low server thread counts, indicating efficient load balancing.

FCFS and **SJF** show higher response and waiting times under high client concurrency or small packet sizes

Throughput improves with **higher client threads** and **larger packet sizes**, and FCFS achieves the best throughput under high packet sizes

Overall, **RR is best for fast responses**, while **FCFS is best for high throughput**, and more server threads help handle larger requests efficiently.

When comparing reads and writes in our **RR** benchmarks, it was seen that reads take longer time than writes, as the server is limited by its packetization parameter while servicing reads.

References

1) https://rapidjson.org/
2) https://github.com/CLIUtils/CLI11
3) https://takeuforward.org/operating-system/scheduling-algorithms
4) https://www.geeksforgeeks.org/operating-systems/round-robin-scheduling-in-operating-system